1-Number of Zeros in a Given Array

Question:

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

PROGRAM:

```
#include <stdio.h>

int count0(int a[], int l, int r, int n)

{    if(l >

r)

      return 0;    int m = (l + r) / 2;    if(a[m] == 0)       return

count0(a, l, m - 1, n) + (m == 0 || a[m-1] == 1 ? n-m : 0);    return

count0(a, m + 1, r, n);

}

int main()

{    int

n;

   scanf("%d", &n);

   int a[n];    for(int i = 0; i < n; i++)

scanf("%d", &a[i]);    printf("%d",

count0(a, 0, n-1, n));    return 0;

}
```

OUTPUT:

	Input	Expected	Got	
✓	5 1 1 1 0 0	2	2	✓

✓	10 1 1 1 1 1 1 1 1 1 1	0	0	✓
✓	8 0 0 0 0 0 0 0 0	8	8	✓
✓	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✓

Passed all tests ! ✓

2-Majority Element

Question :
Given an array `nums` of size `n`, return *the majority element*.
The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]
Output: 3 **Example 2:**

Input: nums = [2,2,1,1,1,2,2]
Output: 2

Constraints:

- n == nums.length
- 1 <= n <= $5 * 10^4$
- -2^{31} <= nums[i] <= $2^{31} - 1$

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

PROGRAM:

```
#include <stdio.h> int
main() {
    int n;
    scanf("%d", &n);     int
nums[n];     for (int i = 0; i
< n; i++) {
        scanf("%d", &nums[i]);
    }
    int candidate = 0;     int
count = 0;     for (int i = 0; i
< n; i++) {         if (count ==
0) {             candidate =
nums[i];
        }
        if (nums[i] == candidate) {
            count++;
} else {
count--;
        }
    }
    printf("%d\n", candidate);
return 0; }
```

OUTPUT:

	Input	Expected	Got	
✓	3 3 2 3	3	3	✓

Passed all tests! ✓

3-Finding Floor Value

Question:

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array
Next n lines Contains n numbers – Elements of an array
Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

PROGRAM:

```
#include <stdio.h>
int f(int a[], int l, int r, int x, int ans)
{   if(l >
r)
    return ans;   int m = (l + r)
/ 2;   if(a[m] <= x)    return
f(a, m + 1, r, x, a[m]);   return
f(a, l, m - 1, x, ans);
}
int main()
{   int n,
x;
  scanf("%d", &n);
```

```
int a[n]; for(int i = 0; i < n;
i++) scanf("%d", &a[i]);
scanf("%d", &x); printf("%d",
f(a, 0, n-1, x, -1));
return 0; }
```

OUTPUT:

	Input	Expected	Got	
✓	6 1 2 8 10 12 19 5	2	2	✓
✓	5 10 22 85 108 129 100	85	85	✓
✓	7 3 5 7 9 11 13 15 10	9	9	✓

Passed all tests! ✓

4-Two Elements sum to x

Problem Statement:
Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as “No”.
Note: Write a Divide and Conquer Solution
Input Format
First Line Contains Integer n – Size of array
Next n lines Contains n numbers – Elements of an array
Last Line Contains Integer x – Sum Value
Output Format
First Line Contains Integer – Element1
Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value “x”)
PROGRAM:

```
#include <stdio.h> int binarySearch(int arr[], int

low, int high, int key)

{

   if (low > high)        return -1;     int mid = (low

+ high) / 2;     if (arr[mid] == key)        return

mid;     else if (arr[mid] > key)        return

binarySearch(arr, low, mid - 1, key);

   else        return binarySearch(arr, mid + 1,

high, key);

}

int findPair(int arr[], int low, int high, int x)

{

   if (low > high)        return 0;     int required = x -

arr[low];     if (binarySearch(arr, low + 1, high,

required) != -1)

   {

      printf("%d\n", arr[low]);

printf("%d\n", required);

      return 1;

   }
```

```
    return findPair(arr, low + 1, high, x);
}
int main()
{
    int n, x;
    scanf("%d", &n);     int
arr[n];     for (int i = 0; i < n;
i++)        scanf("%d",
&arr[i]);     scanf("%d", &x);
if (!findPair(arr, 0, n - 1, x))
printf("No");     return 0;
}
```

OUTPUT:

	Input	Expected	Got	
✓	4 2 4 8 10 14	4 10	4 10	✓
✓	5 2 4 6 8 10 100	No	No	✓

Passed all tests! ✓

5-Implementation of Quick Sort

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

PROGRAM:

```
#include <stdio.h>
void quick(int a[], int l, int r)
{
   if(l >= r) return;
   int i = l, j = r;
   int p = a[(l + r) / 2];
   int t;
   while(i <= j)
   {
      while(a[i] < p)
         i++;
      while(a[j] > p)
         j--;
      if(i <= j)
      {
         t = a[i];
         a[i] = a[j];
         a[j] = t;
         i++;
         j--;
      }
   }
   if(l < j)
      quick(a, l, j);
   if(i < r)
      quick(a, i, r);
```

```
}
int main()
{
   int n;
   scanf("%d", &n);
   int a[n];
   for(int i = 0; i < n; i++)
      scanf("%d", &a[i]);
   quick(a, 0, n - 1);
   for(int i = 0; i < n; i++)
      printf("%d ", a[i]);
   return 0;
}
```

OUTPUT:

	Input	Expected	Got	
✓	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓